

Tạo ASP.NET Core 10 Web API

Mục tiêu bài lab

Sau khi hoàn thành, sinh viên có thể:

1. Tạo project **ASP.NET Core Web API** bằng CLI hoặc Visual Studio
2. Xây dựng REST API quản lý **User** với kiến trúc 3 tầng (Controller – Service – Repository)
3. Lưu trữ dữ liệu trong **List** (không cần database)
4. Bật **Swagger UI** để xem và thử API (OpenAPI) ngay trên trình duyệt

Thời gian: 90 phút **Công cụ:** Visual Studio 2022 (hoặc VS Code), .NET 10 SDK

Phần mềm	Phiên bản	Tải về
.NET SDK	10.0+	dotnet.microsoft.com
Visual Studio	2022 Community	visualstudio.microsoft.com
VS Code (thay thế)	Bất kỳ	code.visualstudio.com

Kiểm tra .NET trước khi bắt đầu: Mở terminal và chạy `dotnet --version`. Kết quả phải là 10.x.x trở lên.

1 Tạo Project Web API

1.1. Bước 1 – Tạo solution và project bằng .NET CLI

Mở terminal (Command Prompt / PowerShell / Terminal) và chạy lần lượt:

Listing 1: Tạo thư mục và project

```
# Tao thu muc goc
mkdir UsersApp
cd UsersApp

# Tao Solution file (quan ly nhieu project)
dotnet new sln -n UsersApp

# Tao project Web API (co OpenAPI / Swagger -- KHONG dung --no-openapi)
dotnet new webapi -n UsersApp.Api -f net10.0

# Chi co mot project Api trong solution
dotnet sln add UsersApp.Api/UsersApp.Api.csproj

# Swagger UI (Swashbuckle) -- can cho giao dien /swagger
cd UsersApp.Api
dotnet add package Swashbuckle.AspNetCore
cd ..
```

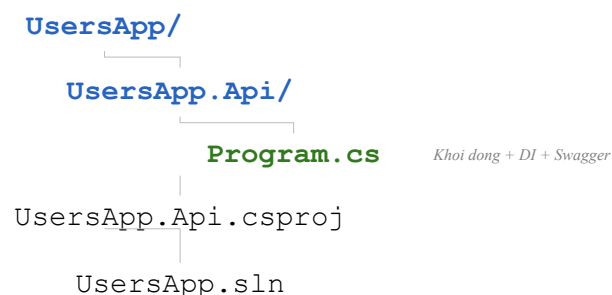
Template webapi mặc định đã kèm tài liệu OpenAPI. Gói **Swashbuckle.AspNetCore** thêm **Swagger UI** tại /swagger. Hướng dẫn phần `Program.cs` bên dưới sẽ đăng ký đầy đủ middleware tương thích với bài lab.

1.2. Bước 2 – Mở project trong Visual Studio / VS Code

Visual Studio 2022: File → Open → chọn file `UsersApp.sln`

VS Code: code `.` trong thư mục `UsersApp`

1.3. Bước 3 – Cấu trúc thư mục sau khi tạo



Khác với Spring Boot – ASP.NET Core không tự tạo thư mục `Controllers/`, `Services/...`.
Sinh viên tự tạo và tổ chức theo kiến trúc mong muốn.

2 Tạo Model và Repository

2.1. Bước 1 – Tạo thư mục tổ chức code

Trong thư mục `UsersApp.Api/`, tạo 3 thư mục con: `Models`, `Repositories`, `Services`, `Controllers`

```
cd UsersApp.Api
mkdir Models Repositories Services Controllers
```

2.2. Bước 2 – Tạo Model – `User.cs`

Tạo file `UsersApp.Api/Models/User.cs`:

[FILE] `Models/User.cs`

```
namespace UsersApp.Api.Models;

/// <summary>
/// Model đại diện cho một người dùng trong hệ thống.
/// Lưu trữ trong List (không dùng database).
/// </summary>
public class User
{
    public long Id { get; set; }
    public string Name { get; set; } = string.Empty;
}
```

2.3. Bước 3 – Tạo Interface Repository – `IUsersRepository.cs`

Tạo file `UsersApp.Api/Repositories/IUsersRepository.cs`:

[FILE] Repositories/IUsersRepository.cs

```
using UsersApp.Api.Models;

namespace UsersApp.Api.Repositories;

/// <summary>
/// Interface định nghĩa "hợp đồng" cho Data layer.
/// Service chi phụ thuộc vào interface này
/// --> để thay implementation (List, EF Core, ...) mà không
///     đổi Service.
/// </summary>
public interface IUsersRepository
{
    IList<User> GetAll();
    User? GetById(long id);    // ? = có thể null (nullable)
    User Add(User user);
}
```

2.4. Bước 4 – Tạo class Implementation – UsersRepository.cs

Tạo file `UsersApp.Api/Repositories/UsersRepository.cs`:

[FILE] Repositories/UsersRepository.cs

```
using UsersApp.Api.Models;

namespace UsersApp.Api.Repositories;

/// <summary>
/// Implementation: lưu dữ liệu trong List (thay thế database)
/// .
/// </summary>
public class UsersRepository : IUsersRepository
{
    // Dữ liệu mẫu -- lưu trong RAM, mất khi restart app
    private readonly List<User> _storage = new()
    {
        new User { Id = 1, Name = "Nguyen Van An" },
        new User { Id = 2, Name = "Tran Thi Binh" },
        new User { Id = 3, Name = "Le Van Cuong" },
    };

    public IList<User> GetAll() => _storage;

    public User? GetById(long id)
        => _storage.FirstOrDefault(u => u.Id == id);

    public User Add(User user)
    {
        // Tự động gán ID theo kích thước hiện tại
    }
}
```

```
        user.Id = _storage.Count + 1;
        _storage.Add(user);
        return user;
    }
}
```

So sánh với Java: `Optional<User>` trong Java tương đương `User?` (nullable) trong C#. Cả hai đều giải quyết vấn đề “không tìm thấy” mà không cần trả `null` trực tiếp.

3 Tạo Service Layer

Tạo file `UsersApp.Api/Services/UsersService.cs`:

[FILE] `Services/UsersService.cs`

```
using UsersApp.Api.Models;
using UsersApp.Api.Repositories;

namespace UsersApp.Api.Services;

/// <summary>
/// Service layer -- xu ly business logic.
/// Phu thuoc vao IUsersRepository (interface), khong phu
/// thuoc impl.
/// </summary>
public class UsersService
{
    private readonly IUsersRepository _usersRepository;

    // Constructor Injection -- giong Spring Boot
    public UsersService(IUsersRepository usersRepository)
    {
        _usersRepository = usersRepository;
    }

    /// <summary>Lay danh sach tat ca user.</summary>
    public IList<User> GetUsers()
        => _usersRepository.GetAll();

    /// <summary>
    /// Tim user theo ID.
    /// Nem KeyNotFoundException neu khong tim thay.
    /// </summary>
    public User GetUserById(long id)
    {
        var user = _usersRepository.GetById(id);
        if (user is null)
            throw new KeyNotFoundException($"User not found
            with id: {id}");
        return user;
    }

    /// <summary>
    /// Them user moi.
    /// Validate: ten khong duoc trong hoac chi co khoang
    /// trang.
    /// </summary>
    public User AddUser(User user)
    {
        if (string.IsNullOrWhiteSpace(user.Name))
```

```
        throw new ArgumentException("Name cannot be empty")
        );

        return _userRepository.Add(user);
    }
}
```

Java (Spring)	C# (.NET)	Tình huống
RuntimeException	KeyNotFoundException	Không tìm thấy entity
IllegalArgumentException	ArgumentException	Dữ liệu đầu vào sai
Optional.orElseThrow()	if (user is null) throw	Kiểm tra null
String.isBlank()	string.IsNullOrWhiteSpace()	Chuỗi rỗng

4 Tạo Controller Layer

Tạo file `UsersApp.Api/Controllers/UsersController.cs`:

[FILE] `Controllers/UsersController.cs`

```
using Microsoft.AspNetCore.Mvc;
using UsersApp.Api.Models;
using UsersApp.Api.Services;

namespace UsersApp.Api.Controllers;

[ApiController]
[Route("[controller]")] // Route tự động = /Users (tên class
bo chu "Controller")
public class UsersController : ControllerBase
{
    private readonly UsersService _userService;

    public UsersController(UsersService userService)
    {
        _userService = userService;
    }

    // GET /users
    [HttpGet]
    public IActionResult GetUsers()
    {
        var users = _userService.GetUsers();
        return Ok(users); // HTTP 200 + JSON
    }

    // GET /users/{id}
    [HttpGet("{id}")]
    public IActionResult GetUserById(long id)
    {
        try
        {
            var user = _userService.GetUserById(id);
            return Ok(user); // HTTP 200
        }
        catch (KeyNotFoundException)
        {
            return NotFound(); // HTTP 404
        }
    }

    // POST /users
    [HttpPost]
    public IActionResult AddUser([FromBody] User user)
    {
```

```
        try
        {
            var saved = _userService.AddUser(user);
            // HTTP 201 + Location header tro den GET /users/{
            id}
            return CreatedAtAction(nameof(GetUserById),
                                   new { id = saved.Id },
                                   saved);
        }
        catch (ArgumentException)
        {
            return BadRequest("Name cannot be empty"); // HTTP
            400
        }
    }
}
```

HTTP	URL	Action	Status
GET	/users	Lấy tất cả user	200 OK
GET	/users/{id}	Lấy theo ID	200 / 404
POST	/users	Thêm user mới	201 Created

5 Cấu hình Program.cs – Đăng ký Dependency Injection

Đây là bước **quan trọng nhất** khác biệt với Spring Boot. Thay vì dùng annotation `@Service`, `@Repository`, ASP.NET Core yêu cầu đăng ký thủ công trong `Program.cs`.

Mở file `UsersApp.Api/Program.cs` và thay toàn bộ nội dung:

[FILE] Program.cs -- Entry point + DI Container

```
using UsersApp.Api.Repositories;
using UsersApp.Api.Services;

var builder = WebApplication.CreateBuilder(args);

//
=====

// DANG KY DEPENDENCY INJECTION
// (tuong duong @Repository, @Service trong Spring Boot)
//
=====

// AddScoped: tao moi object theo moi HTTP request
builder.Services.AddScoped<IUsersRepository, UsersRepository>();
builder.Services.AddScoped<UsersService>();

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

//
=====

// MIDDLEWARE PIPELINE
//
=====

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI(options =>
    {
        options.SwaggerEndpoint("/swagger/v1/swagger.json", "
            Users API v1");
        options.RoutePrefix = "swagger"; // https://localhost:
            XXXX/swagger
    });
}
```

```
}  
  
app.UseHttpsRedirection();  
app.MapControllers();  
  
app.Run();
```

• Giải thích vòng đời (Lifetime) của DI

Lifetime	Tạo object mới khi nào	Dùng cho
AddScoped	Mỗi HTTP request	Repository, Service (thông thường)
AddTransient	Mỗi lần inject	Service nhẹ, stateless
AddSingleton	Khởi động app, dùng mãi	Cache, Config, Logger

Quy tắc nhớ: Dùng AddScoped cho hầu hết Repository và Service. Tương đương với @Service + @Repository trong Spring Boot (Spring Boot mặc định là Scoped cho mỗi request).

6 Chạy Thử Ứng Dụng

6.1. Bước 1 – Chạy ứng dụng

Listing 2: Chạy từ terminal

```
cd UsersApp.Api
dotnet run
```

Listing 3: Output khi khởi động thành công

```
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:5001
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
```

6.2. Bước 2 – Kiểm tra API

Mở trình duyệt và truy cập:

```
http://localhost:5000/users
```

Kết quả JSON trả về:

Listing 4: Response JSON

```
[
  { "id": 1, "name": "Nguyen Van An" },
  { "id": 2, "name": "Tran Thi Binh" },
  { "id": 3, "name": "Le Van Cuong" }
]
```

6.3. Bước 3 – Gửi POST bằng curl

Listing 5: Thêm user mới bằng curl

```
curl -X POST http://localhost:5000/users \
  -H "Content-Type: application/json" \
  -d '{"name": "Pham Thi Dao"}'
```

Nếu lỗi SSL certificate: Chạy `dotnet dev-certs https --trust` một lần để tin tưởng certificate phát triển, hoặc dùng `http://` thay `https://`.

7 Swagger UI – Tài liệu và thử API trên trình duyệt

Swagger UI là giao diện web đọc file mô tả OpenAPI (JSON) và hiển thị danh sách endpoint, kiểu dữ liệu request/response. Bạn có thể **gửi thử** GET/POST trực tiếp mà không cần Postman hay curl.

7.1. Điều kiện

Đã cài Swashbuckle.AspNetCore, trong `Program.cs` có `AddEndpointsApiExplorer`, `AddSwaggerGen`, và trong môi trường **Development** có `UseSwagger + UseSwaggerUI` như phần trên.

`UseSwagger/UseSwaggerUI` được đặt trong `if (app.Environment.IsDevelopment())` để **không** lộ tài liệu API chi tiết trên môi trường production. Khi deploy thật cần cấu hình bảo mật riêng nếu vẫn muốn mở Swagger.

7.2. Bước 1 – Mở Swagger UI

Giữ `dotnet run` đang chạy. Trên terminal, ASP.NET Core in địa chỉ lắng nghe (ví dụ `http://localhost` và `https://localhost:5001`).

Mở trình duyệt, truy cập (thay cổng cho đúng với máy bạn):

Listing 6: URL Swagger UI (thường dùng HTTPS)

```
https://localhost:5001/swagger
```

Hoặc nếu chỉ dùng HTTP:

```
http://localhost:5000/swagger
```

Trang hiển thị tiêu đề kiểu **Swagger UI**, danh sách nhóm API (theo controller) và từng thao tác.

7.3. Bước 2 – Thử GET /users

1. Mở rộng nhóm **Users** (hoặc tên controller của bạn).
2. Chọn GET `/users` → **Try it out** → **Execute**.
3. Xem **Response body** – JSON danh sách user; **Response code** phải là 200.

7.4. Bước 3 – Thử POST /users

1. Chọn POST `/users` → **Try it out**.
2. Trong ô **Request body**, nhập JSON, ví dụ:

```
{
  "name": "Pham Thi Dao"
}
```

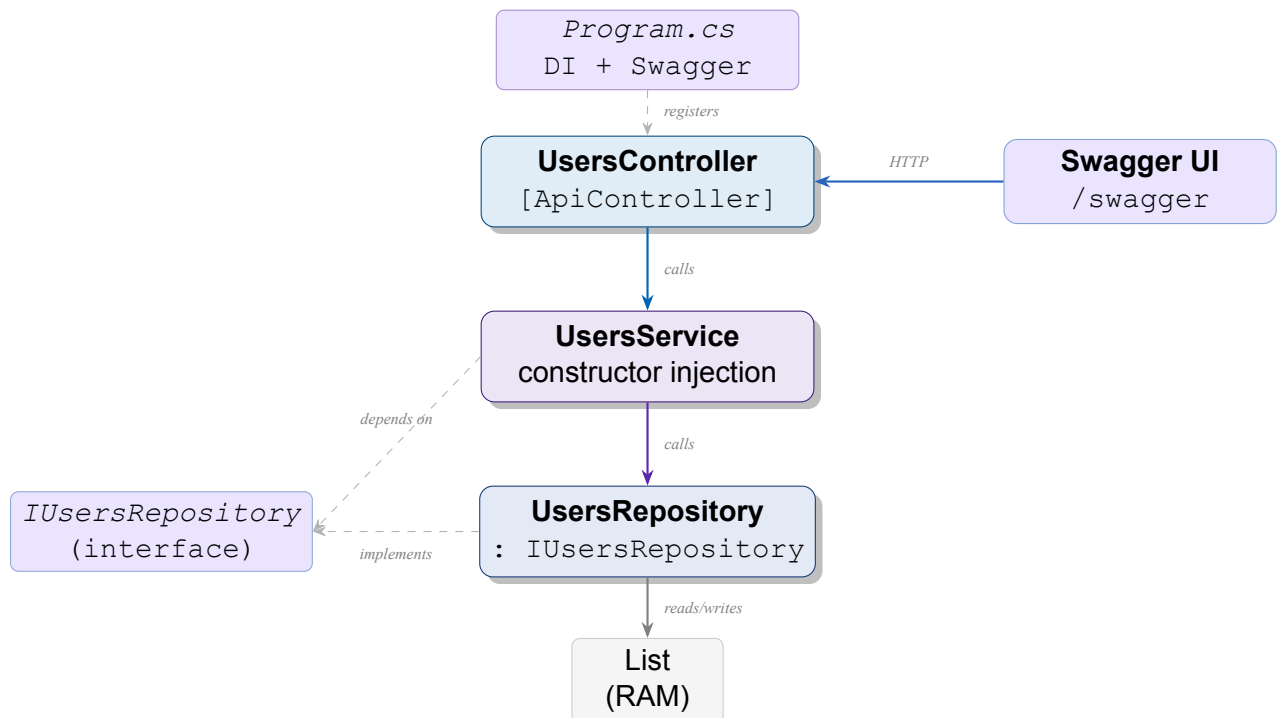
3. **Execute** – kiểm tra 201 Created và body trả về có id, name.

7.5. Gợi ý xử lý sự cố

- **404 tại /swagger:** Kiểm tra đã gọi `UseSwagger/UseSwaggerUI`, đang chạy ở **Development**, và URL đúng (`/swagger`, không thiếu tiền tố).
- **Trống danh sách endpoint:** Đảm bảo `MapControllers()` và controller có `[ApiController]` + `route [controller]`.
- **Cổng không khớp:** Xem dòng `Now listening on:` trong console sau `dotnet run`; dùng đúng `http` hoặc `https` tương ứng.

File mô tả OpenAPI (JSON) thường có đường dẫn `/swagger/v1/swagger.json` – có thể mở trực tiếp trên trình duyệt hoặc nhập vào Postman (*Import from URL*) khi cần chia sẻ tài liệu với nhóm.

8 Tổng kết – Sơ đồ Project



8.1. Bảng tổng kết kiến thức

Thành phần	Keyword / Attribute	Vai trò
Model	class thuần	POCO – chứa dữ liệu
Repository	IUsersRepository (interface)	Định nghĩa Data layer
Repository Impl	: IUserRepository	Cài đặt cụ thể (dùng List)
Service	UserService	Business logic
Controller	[ApiController]	Xử lý HTTP request
DI Registration	AddScoped<I, Impl>()	Đăng ký trong Program.cs
OpenAPI doc	AddSwaggerGen()	Sinh mô tả API (JSON)
Swagger UI	UseSwagger(), UseSwaggerUI()	Trang thử API tại /swagger

Câu hỏi tự kiểm tra:

1. Tại sao **UserService** phụ thuộc vào **IUsersRepository** (interface) chứ không phải **UserRepository** (class)?
2. **AddScoped**, **AddTransient**, **AddSingleton** khác nhau như thế nào?

3. Swagger UI lấy thông tin endpoint từ đâu (OpenAPI JSON) và vì sao nên giới hạn Swagger ở môi trường Development?
4. Khác biệt giữa thử `POST /users` trong Swagger UI và gọi `curl` là gì?